Understanding Set Operators in SQL: UNION, UNION ALL, INTERSECT, and EXCEPT

SQL provides several set operators that allow you to combine the results of two or more `SELECT` statements. These operators are useful for performing complex queries and data manipulation. This document will explain four key set operators: `UNION`, `UNION ALL`, `INTERSECT`, and `EXCEPT`, along with practical examples.

Let's consider two hypothetical tables, `EmployeesA` and `EmployeesB`, to illustrate the examples.

Table: EmployeesA

EmployeeID	Name	Department
1	Alice	HR
2	Bob	Sales
3	Charlie	Marketing

Table: EmployeesB

EmployeeID	Name	Department
2	Bob	Sales
4	David	IT
5	Eve	HR

UNION

The `UNION` operator combines the result sets of two or more `SELECT` statements and removes duplicate rows. For `UNION` to work, the following conditions must be met:

- Each `SELECT` statement within the `UNION` must have the same number of columns.
- The columns must have similar data types.
- The columns in each `SELECT` statement must be in the same order.

Example:

To get a list of all unique employees from both `EmployeesA` and `EmployeesB`:SELECT EmployeeID, Name, Department

FROM EmployeesA

UNION

SELECT EmployeeID, Name, Department

FROM EmployeesB;

Result:

EmployeeID	Name	Department
1	Alice	HR
2	Bob	Sales
3	Charlie	Marketing
4	David	IT
5	Eve	HR

Notice that "Bob" (EmployeeID 2), who appears in both tables, is listed only once in the result because `UNION` removes duplicates.

UNION ALL

The `UNION ALL` operator combines the result sets of two or more `SELECT` statements, but unlike `UNION`, it retains duplicate rows. The same conditions regarding the number of columns, data types, and order apply as with `UNION`.

Example:

To get a list of all employees from both `EmployeesA` and `EmployeesB`, including duplicates:SELECT EmployeeID, Name, Department

FROM EmployeesA

UNION ALL

SELECT EmployeeID, Name, Department

FROM EmployeesB;

Result:

EmployeeID	Name	Department
1	Alice	HR
2	Bob	Sales
3	Charlie	Marketing
2	Bob	Sales
4	David	IT
5	Eve	HR

Here, "Bob" (EmployeeID 2) appears twice because `UNION ALL` includes all rows from both result sets without filtering for duplicates.

INTERSECT

The `INTERSECT` operator returns only the rows that are present in both `SELECT` statements. In other words, it returns the common rows between two result sets.

Example:

To find employees who are present in both `EmployeesA` and `EmployeesB`:SELECT EmployeeID, Name, Department

FROM EmployeesA

INTERSECT

SELECT EmployeeID, Name, Department

FROM EmployeesB;

Result:

EmployeeID	Name	Department
2	Bob	Sales

The only row common to both tables is "Bob" (EmployeeID 2).

EXCEPT

The `EXCEPT` operator returns all distinct rows from the first `SELECT` statement that are not found in the second `SELECT` statement.

Example:

To find employees who are in `EmployeesA` but not in `EmployeesB`:SELECT EmployeeID, Name, Department

FROM EmployeesA

EXCEPT

SELECT EmployeeID, Name, Department

FROM EmployeesB;

Result:

EmployeeID	Name	Department
1	Alice	HR
3	Charlie	Marketing

Alice and Charlie are in `EmployeesA` but not in `EmployeesB`.

Example (Reversed):

To find employees who are in `EmployeesB` but not in `EmployeesA`:SELECT EmployeeID, Name, Department

FROM EmployeesB

EXCEPT

SELECT EmployeeID, Name, Department

FROM EmployeesA;

Result:

EmployeeID	Name	Department
4	David	IT
5	Eve	HR

David and Eve are in `EmployeesB` but not in `EmployeesA`.

These set operators are powerful tools for querying and combining data from multiple tables, offering flexibility in how you manage and analyze your database information.